

Mettre à jour un projet avec Git et GitHub

Bastien Marguet

July 1, 2025

Pourquoi utiliser Git et GitHub ?

- Git permet de suivre l'évolution de ton projet fichier par fichier.
- GitHub héberge ton dépôt en ligne pour le partager ou le publier (ex: site GitHub Pages).
- La combinaison rend le déploiement de modifications rapide, sécurisé et collaboratif.

1. Se placer dans le bon dossier

Commande :

```
cd /chemin/vers/ton/dossier
```

- Ouvre ton terminal (ou Git Bash).
- Navigue jusqu'au dossier local où se trouvent tes fichiers du projet.
- Vérifie que tu vois bien tes fichiers en tapant `ls` ou `dir`.

2. Vérifier les modifications

Commande :

```
git status
```

- Affiche quels fichiers sont modifiés, ajoutés ou supprimés.
- Vérifie avant de préparer ton commit.
- Important pour éviter d'oublier un fichier important.

3. Préparer les fichiers à envoyer

Commande :

```
git add .
```

- `git add .` ajoute **tous** les fichiers modifiés au prochain commit.
- Tu peux être plus précis : `git add index.html style.css`.
- Bonne pratique : vérifier après avec `git status`.

4. Créer un commit explicite

Commande :

```
git commit -m "Ton message clair"
```

- Décris ce que tu as fait : « Mise à jour responsive », « Ajout du menu burger ».
- Évite les messages trop vagues comme « update ».
- Chaque commit est une sauvegarde cliquable.

5. Envoyer en ligne (push)

Commande :

```
git push origin main
```

- push = envoyer les modifications vers GitHub.
- origin = le nom du dépôt distant.
- main = ta branche principale (parfois master).
- Après le push : ton site ou ton projet est mis à jour en ligne.

Conseils pour s'adapter à chaque cas

- Toujours vérifier le nom de ta branche : `main` ou `master`.
- Vérifie l'URL du dépôt distant : `git remote -v`.
- Si plusieurs personnes travaillent dessus, utilise `git pull` pour récupérer avant de pousser.
- Pour les sites GitHub Pages : vérifier sur <https://username.github.io/repo>.

Résumé : cycle basique

- ➊ Aller dans le bon dossier.
- ➋ Vérifier les changements.
- ➌ Ajouter les fichiers.
- ➍ Faire un commit avec un message clair.
- ➎ Pousser sur GitHub.

Ton projet est à jour et versionné !

'git status' : que fait-il vraiment ?

- '**git status**' regarde uniquement l'état local du projet.
- Il compare tes fichiers réels avec l'historique Git (.git/).
- Pas de connexion Internet : tout se passe sur ton disque.
- Utile pour savoir :
 - Ce qui est modifié.
 - Ce qui est prêt à être sauvegardé (staged).
 - Ce qui reste à ajouter.

Où Git enregistre l'historique ?

- Chaque projet Git contient un dossier caché `.git/`.
- Ce dossier stocke :
 - L'historique de tous les commits.
 - L'adresse du dépôt distant (GitHub, GitLab...).
 - Les branches locales et distantes.
- '**git status**' lit seulement ce qu'il y a ici.

'git remote' : le lien vers GitHub

- Un '**remote**' est une adresse vers un dépôt en ligne.
- Par convention, le dépôt principal s'appelle `origin`.
- Quand tu clones :

Exemple

```
git clone https://github.com/user/mon-projet.git
```

- Git enregistre automatiquement :

```
origin = https://github.com/user/mon-projet.git
```

Vérifier ton 'remote'

Commande :

```
git remote -v
```

- Montre les URLs configurées pour fetch (récupérer) et push (envoyer).
- Exemple de sortie :

```
origin https://github.com/user/mon-projet.git (fetch)  
origin https://github.com/user/mon-projet.git (push)
```

- Vérifie toujours ton 'remote' avant un 'push'.

Modifier ou ajouter un 'remote'

- Pour changer l'URL :

```
git remote set-url origin https://github.com/...
```

- Pour ajouter un 'remote' si besoin :

```
git remote add origin https://github.com/...
```

- Pour supprimer :

```
git remote remove origin
```

Résumé : 'status' vs 'remote'

- **'git status'** = vérifie l'état local.
- **'remote'** = configure où envoyer et récupérer.
- `git push origin main` :
 - Utilise le 'remote' nommé origin.
 - Pousse sur la branche main sur GitHub.
- Toujours vérifier `remote -v` avant de pousser.